算法实验平台的软件架构设计[1]

内容提要：在算法实验平台的用例模型和分析类模型等需求模型基础上，给出平台的软件架构的概念设计和精化设计过程。架构概念设计给出了关键需求辨识和软件架构初创的方法和结果；而架构精化设计则从逻辑视图、开发视图、数据视图和物理视图阐述运用设计知识精化软件架构的过程和结果。

关键词：算法实验平台；软件体系结构；软件架构设计；软件架构设计过程

1 引言

电子信息（软件工程）专业重点培养能够系统地使用软件工程学科的基础理论和先进技术，解决软件开发领域实际应用问题的专门人才。而软件体系结构作为一门核心的专业必修课，主要讲授在开发复杂软件系统中，架构设计的原理、方法与技术，对于达成专业培养目标起着举足轻重的作用。

软件工程界公认只有在全面掌握各种经过充分实践考验的设计原则和设计模式等架构知识前提下，通过深入洞察待建软件系统的需求分析成果，并利用迭代演进方式才能设计出高质量的软件架构。考虑到架构设计过程所兼具的理论性和实践性给提升软件体系结构教学成效带来的挑战，《专业学位研究生核心课程指南（试行）》给出了实施案例教学的建议。因而，课程组通过构建案例，创设理解架构设计问题和运用架构知识的情境，并借助案例让学生真实地经历和体验架构设计的基本活动。

2 背景介绍

算法实验平台的主要用户包括：平台管理者、算法设计者和算法实验者。各类用户可使用算法实验平台提供的各项功能。具体地：

（1） 平台管理者：通过使用用户管理和账号管理功能，实现平台管理；

（2） 算法实现者：通过使用问题实例（用于指定算法的输入数据）和算法库管理功能，实现对比算法信息及对应问题实例的入库登记和更新管理；

（3） 算法实验者：通过制订、运行和查看执行计划，实现在选定的问题实例下，按指定配置运行一组对比算法，并能查看各算法的执行结果。

此外，算法实验平台还为算法实现者提供标准接口，以使各种算法可以入库并能被平台调用运行；而当运行执行计划正常或异常结束时，算法实验平台还能通过邮件和微信即时通知算法实验者，以满足执行计划运行过程中无人值守的需求。

[1] 倪友聪、杜欣、肖如良、陈明平、应时、陈高琳，福建师范大学计算机与网络空间安全学院，案例是课程组开发；版权归课程组所有，案例只用于教学目的，不对案例所涉及人员与单位做出任何评判。

3 算法实验平台的需求分析模型

为了创设架构设计情境，课程组开展了算法实验平台的需求分析工作。通过构建需求模型，捕获和规约了算法实验平台的功能性需求和非功能性需求。

3.1 功能性需求

从用例模型和分析类模型描述具体的功能需求。

3.1.1 用例模型

用例模型规约了外部执行者（Actor）使用算法实验平台的功能。

（1）执行者及其关系

算法实验平台包括算法实验者、算法设计者、平台管理员、微信系统和邮件系统等外部执行者，如图 1 所示。前三者具有泛化关系，即平台管理员可执行算法设计者和算法实验者所能使用的功能。

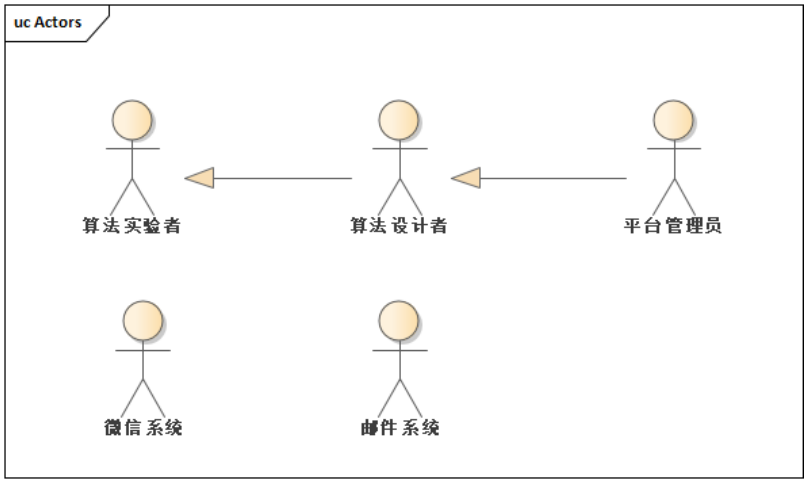

图 1 算法实验平台中外部执行者及其之间关系

（2）用例图

根据功能内聚原则将算法实验平台划分为问题实例管理、算法库管理、执行计划管理、算法结果保存和平台管理等子系统。下面分别给出各子系统用例图，以反映外部执行者使用子系统功能情况。

① 问题实例管理

问题实例管理子系统的用例图如图 2 所示。

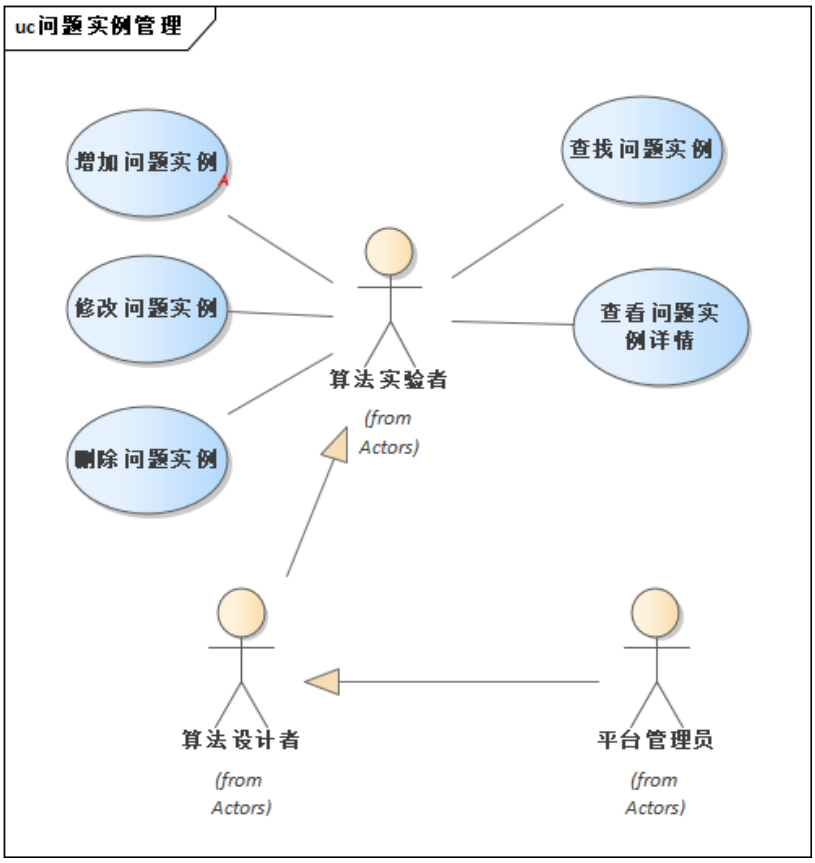

图 2 问题实例管理子系统的用例图

② 算法库管理

算法库管理子系统的用例图如图 3 所示。

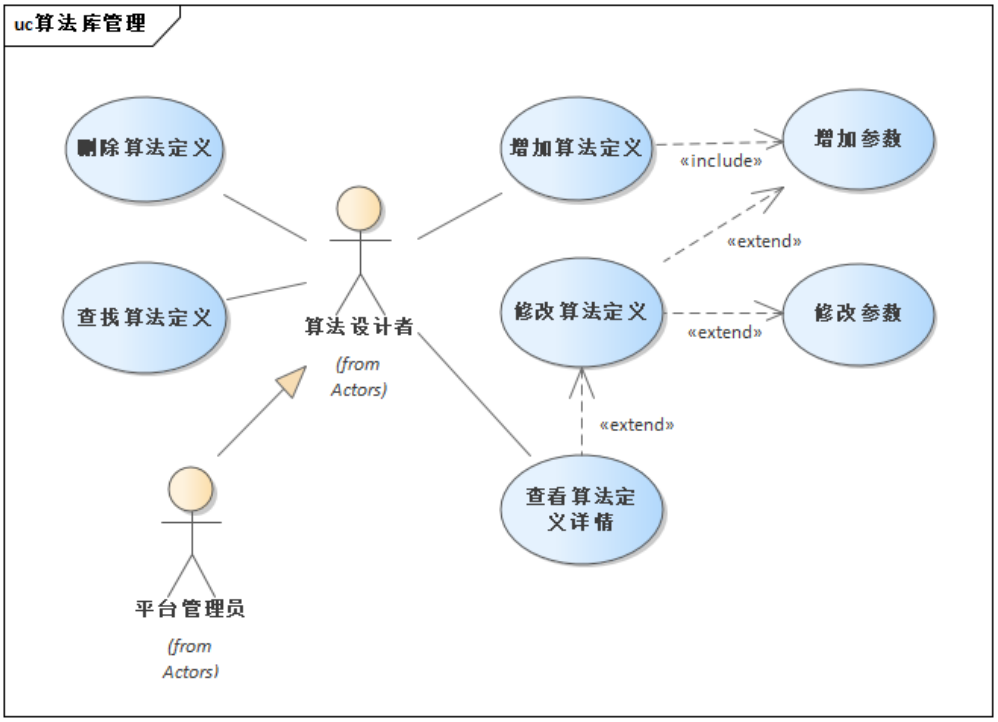

图 3 算法库管理子系统的用例图

③ 执行计划管理

执行计划管理子系统的用例图如图 4 所示。

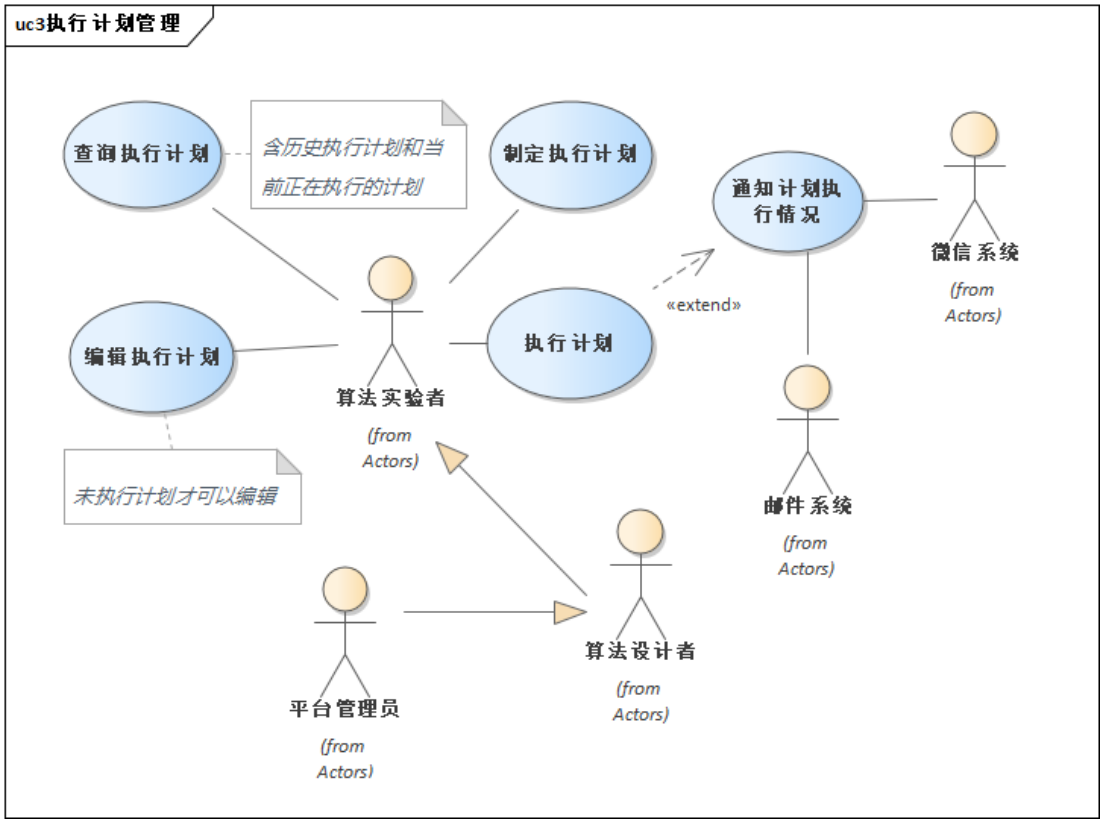

图 4 执行计划管理子系统的用例图

④ 算法结果保存

算法结果保存子系统的用例图如图 5 所示。

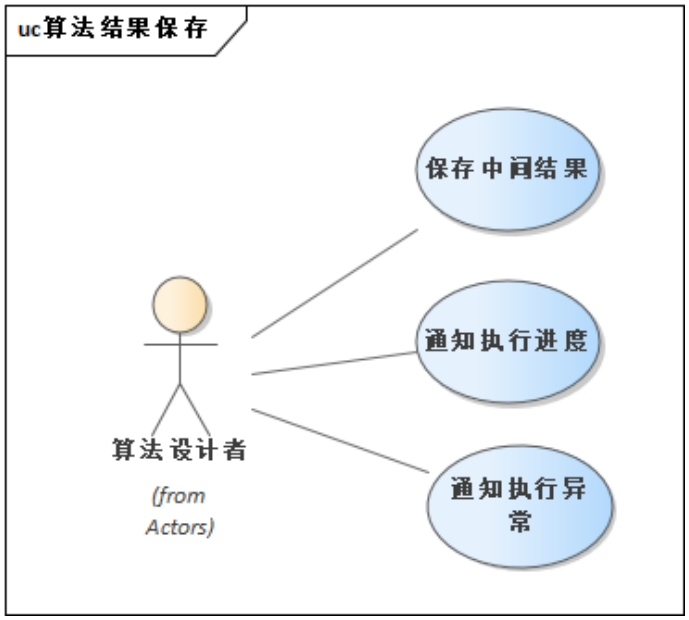

图 5 算法结果保存子系统的用例图

⑤ 平台管理

平台管理子系统的用例图如图 6 所示。

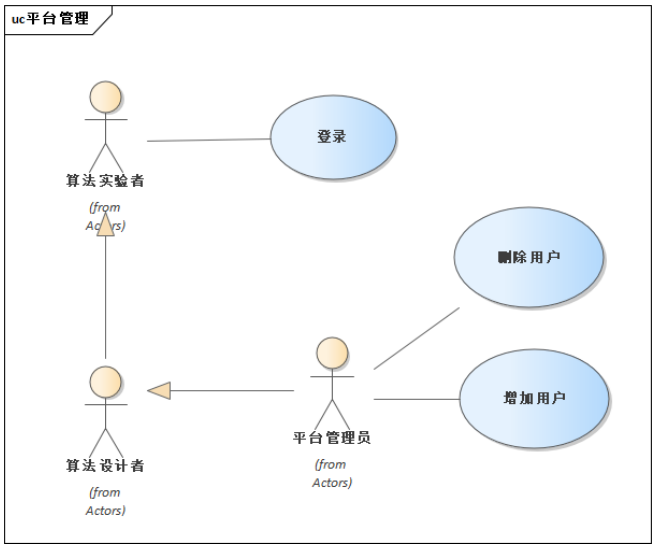

图 6 平台管理子系统的用例图

3.1.2 分析类模型

下面分别给出问题实例管理、算法库管理、执行计划管理、算法结果保存和平台管理等子系统的分析类图，以建模业务概念及其之间关系。

（1）问题实例管理

问题实例管理涉及实例及其分类目录的概念及其关系，如图 7 所示。

（2）算法库管理

算法库管理涉及算法信息及其配置参数的概念及其关系，如图 8 所示。

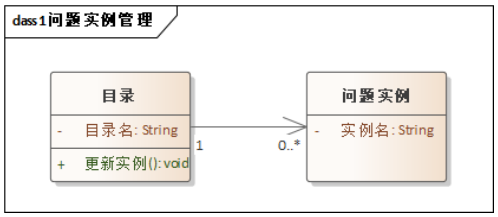

图 7 问题实例管理子系统的分析类图

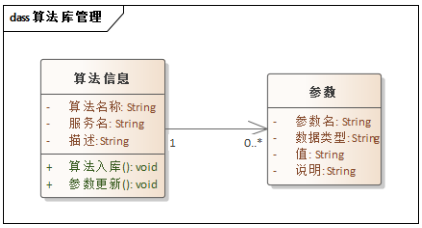

图 8 算法库管理子系统的分析类图

（3）执行计划管理

执行计划管理涉及执行计划、算法运行信息及运行参数三个概念及其之间关系，如图 9 所示。

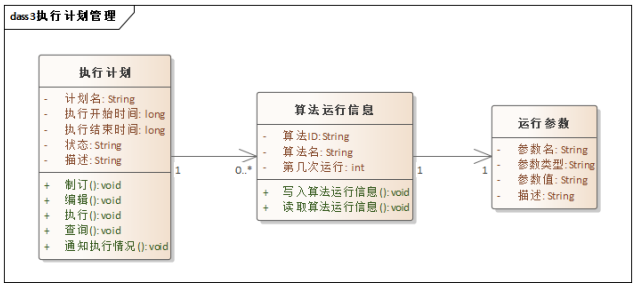

图 9 执行计划管理子系统的分析类图

（4）算法结果保存

算法结果保存涉及计划执行结果、迭代结果和单次运行结果等概念及其之间关系，如图 10 所示。

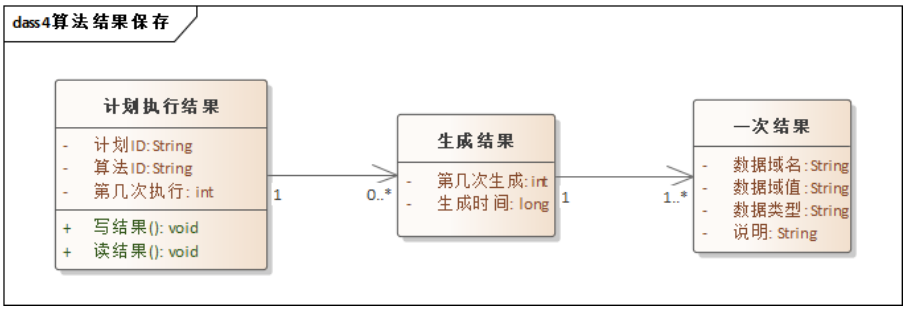

图 10 算法结果保存子系统的分析类图

（5）平台管理

平台管理仅涉及用户信息业务概念，如图 11 所示。

图 11 平台管理子系统的分析类图

3.2 非功能性需求

算法实验平台的非功能性需求主要涉及可扩展性、性能和可靠性三种质量需求。

可扩展性需求：算法实验平台应为算法实现者提供标准接口，以使各种不同的算法可以入库并能被平台调用运行；

性能需求：算法借助平台服务在运行过程中保存结果的时间开销尽可能地小；结果保存尽量不阻塞算法的执行；

可靠性需求：支持算法的多个实例运行，当平台依据执行计划调用算法执行时，能够有效地防止故障。

算法实验平台的需求模型为开展架构设计创设了具体的和多样性的设计情境。

4 算法实验平台的架构设计过程

算法实验平台的架构设计过程主要包括概念设计和架构精化设计。

4.1 概念设计

概念设计主要由关键需求辨识和软件架构初创两个步骤构成。

4.1.1 关键需求辨识

基于辨识方法/辨识依据，表 1 给出了算法实验平台的关键需求。其中：前 5 行与最后 2 行分别对应关键的功能性需求项和非功能性需求项。

表 1 算法实验平台的关键需求辨识

序号	辨识方法/辨识依据	关键需求
1	核心或基础性功能：其他功能依赖于这些功能，或仅当这些功能实现之后其他功能才有意义	增加问题实例、增加算法定义
2	为实现对外接口所必需的支持功能：对外接口的实现依赖于这些功能需求项的实现	通知执行计划情况（依赖于外部邮件接口和微信接口）
3	实现难度较大、实现风险较高的功能：尽早探索高难度、高风险功能的设计方案	通知执行计划情况（需利用微信接口）、运行执行计划（需要平台自动调用设计者实现的算法）
4	最能体现待开发软件特色的功能需求	算法中间结果保存（在算法实现中可利用平台提供的标准服务将执行结果保存至数据存储中）
5	对用户满意度的影响大的功能需求	查询执行计划（按需查看已运行计划的状态）
6	影响用户接受目标软件系统的质量需求	可扩展性需求
7	实现难度较大、实现风险较高的质量需求	性能需求

4.1.2 软件架构初创

体系结构初创的任务是依据关键需求集，从逻辑视图建立与具体软件实现技术无关的初始架构。细化问题实例管理、算法库管理、执行计划管理和算法结果保存等子系统分析类模型，构建案例的初始架构。

（1）问题实例管理

问题实例管理子系统类图细化结果如图 12 所示。

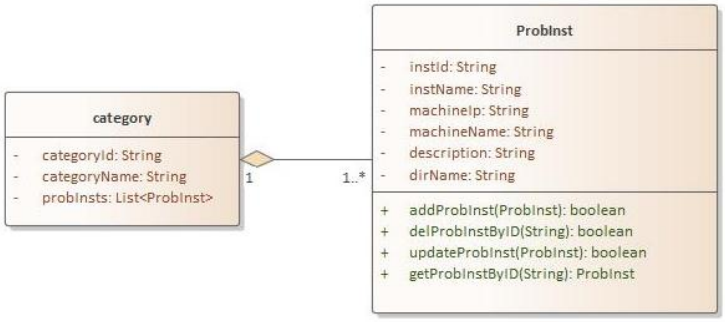

图 12 问题实例管理子系统类图的细化结果

（2）算法库管理

算法库管理子系统类图细化结果如图 13 所示。

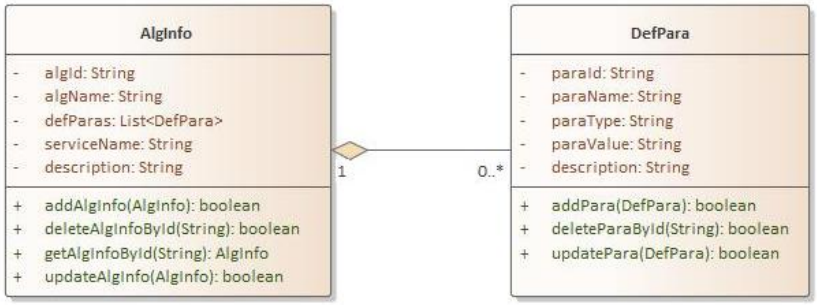

图 13 算法库管理子系统类图的细化结果

（3）执行计划管理

执行计划管理子系统类图细化结果如图 14 所示。

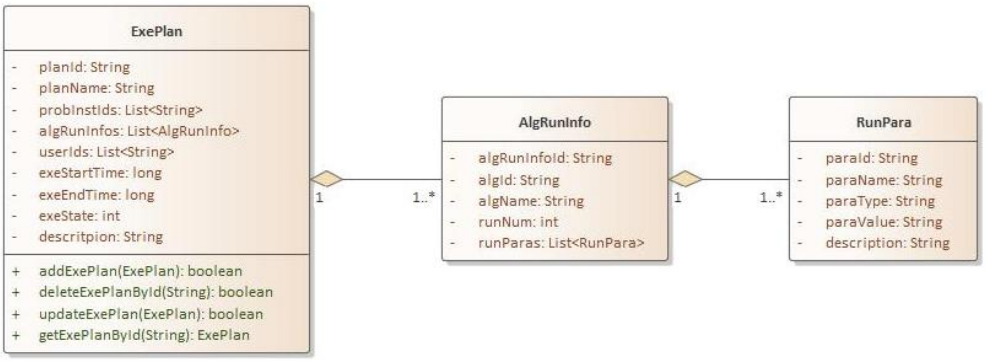

图 14 执行计划管理子系统类图的细化结果

（5）算法执行结果保存

算法结果保存管理子系统类图细化结果如图 15 所示。

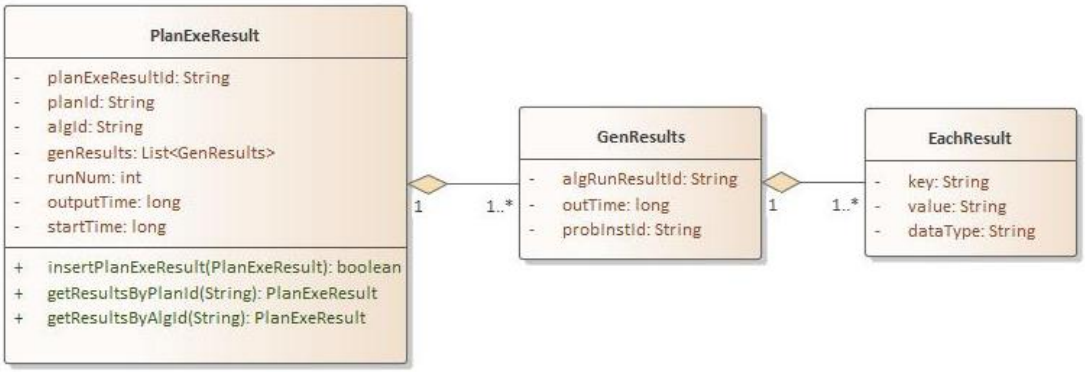

图 15 算法结果保存子系统类图的细化结果

4.2 架构精化设计

架构精化的主要目标是将概念架构精化成为全面的、设计适度的构架。架构精化基于需求模型，并主要围绕逻辑、开发、数据和物理五种视图展开算法实验平台的架构。

4.2.1 逻辑视图

充分应用设计知识并复用已有的开源框架，从逻辑视图精化架构。从而进一步明确定义模块接口。在案例中，采用分层架构模式并复用 Spring Cloud 开源框架，精化得到图 16 所示的总体架构。具体而言：

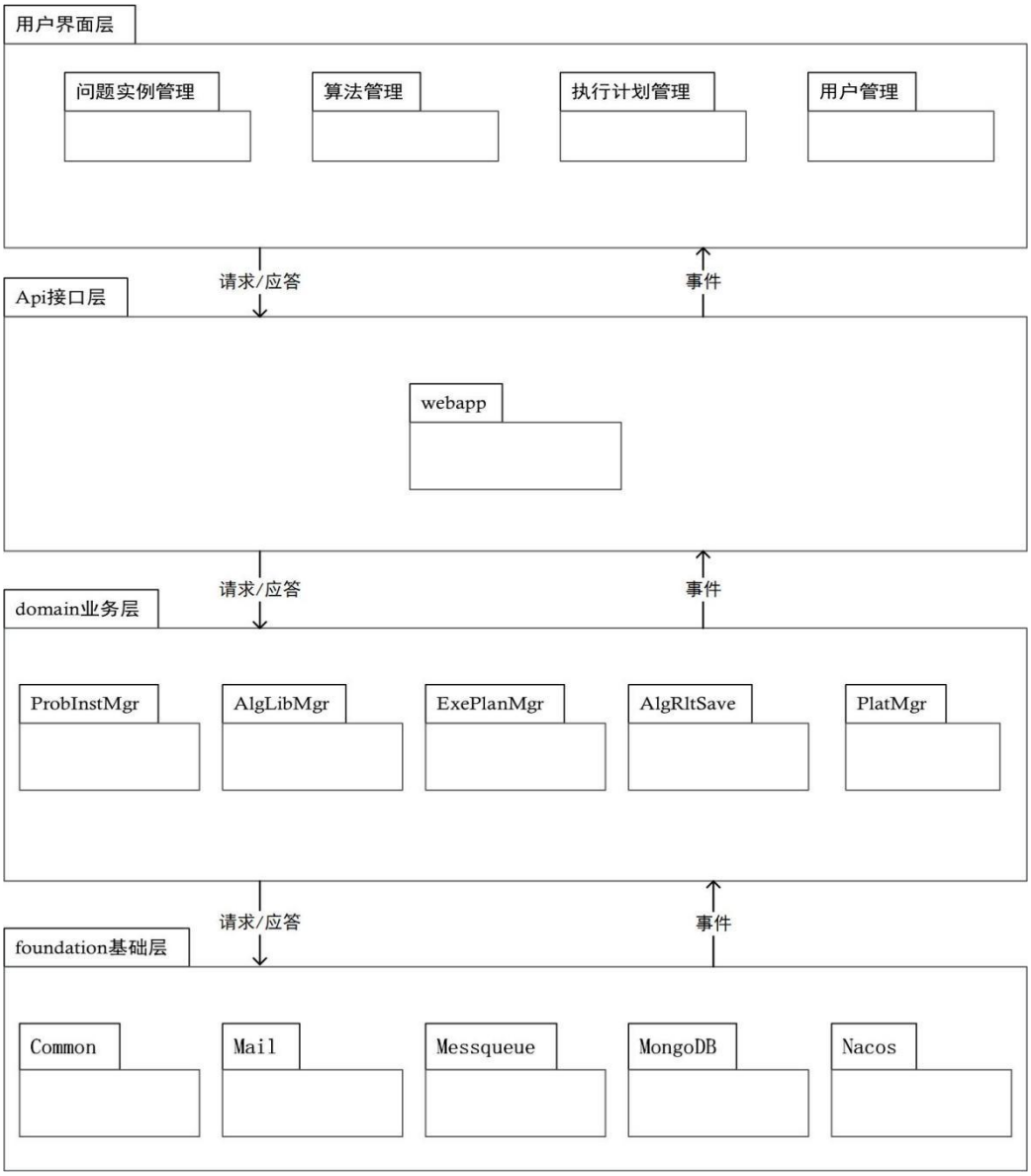

图 16 算法实验平台精化后的总体架构

（1）foundation 基础层

foundation 层主要包含 Common、Mail、Messqueue、 Nacos 和 MongoDB 五

个构件。Common 用于封装通用的公共功能；Mail 提供邮件服务；Messqueue 用于提供构件间异步消息通信以及微信消息访问功能；Nacos 提供服务注册和服务发现等服务管理功能；MongoDB 构件用于封装对 MongoDB 数据库的访问与配置功能。

（2）domain 业务层

domain 层主要包含问题实例管理、算法库管理、执行计划管理、算法结果保存和平台管理五个子系统。各子系统均基于 SpringCloud 框架并采用 Service（服务）、DAO（数据访问）和 Entity（业务实体）设计模式。其精化结果分别由图 17、图 18、图 19、图 20 和图 21 给出。

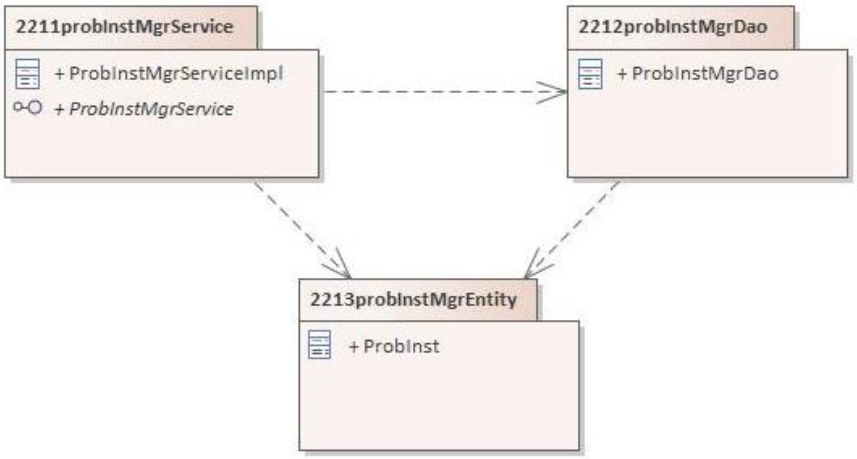

图 17 问题实例管理子系统的精化

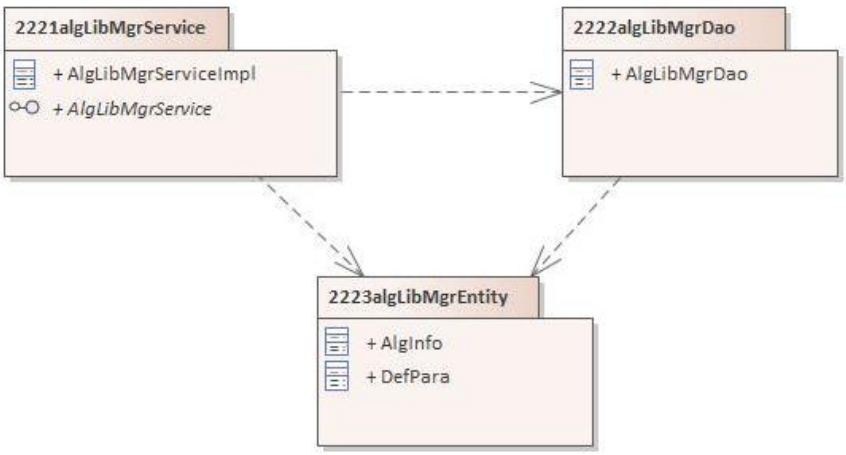

图 18 算法库管理子系统的精化

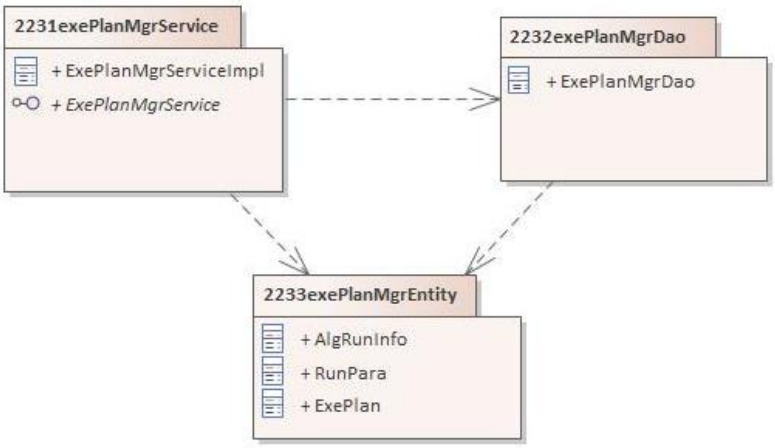

图 19 执行计划管理子系统的精化

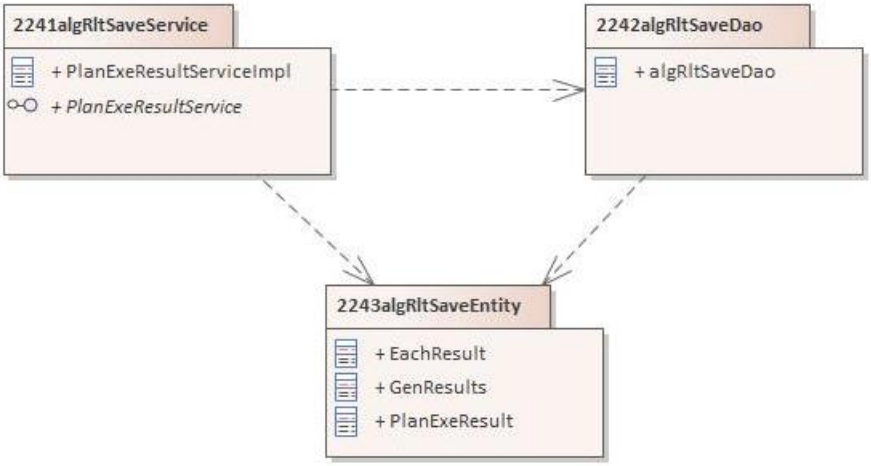

图 20 算法结果保存子系统的精化

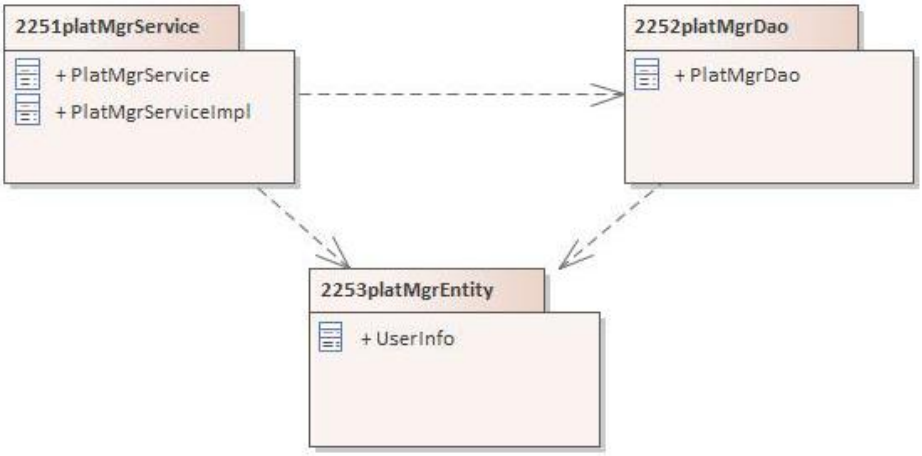

图 21 平台管理子系统的精化

对于算法入库和制订执行计划两项实现较为复杂的功能，图 22 和图 23 给出对应的顺序图，以反映构件之间如何协作完成相应的功能。

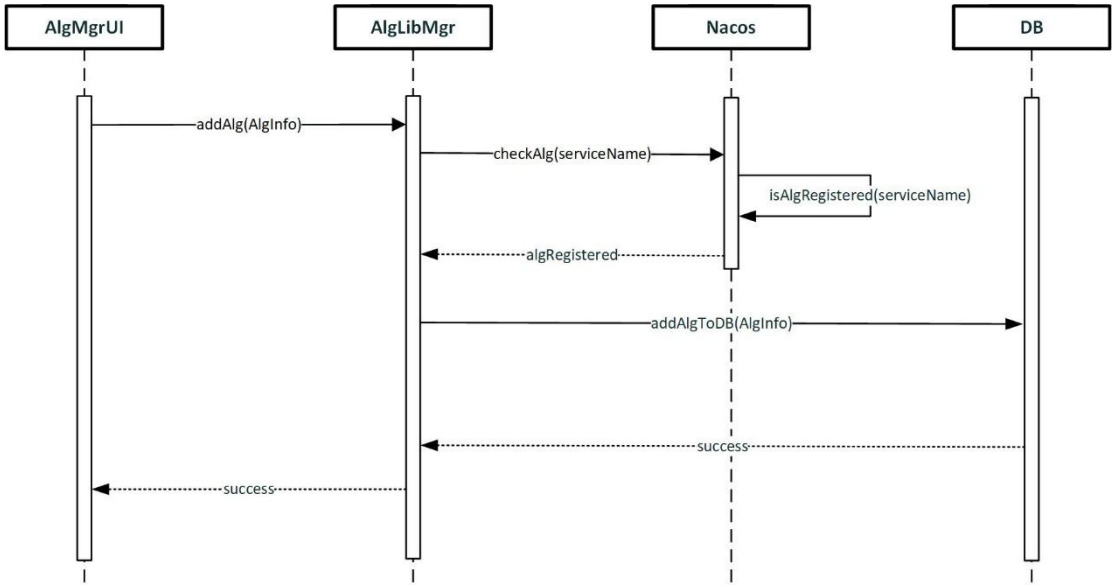

图 22 算法入库功能对应的顺序图

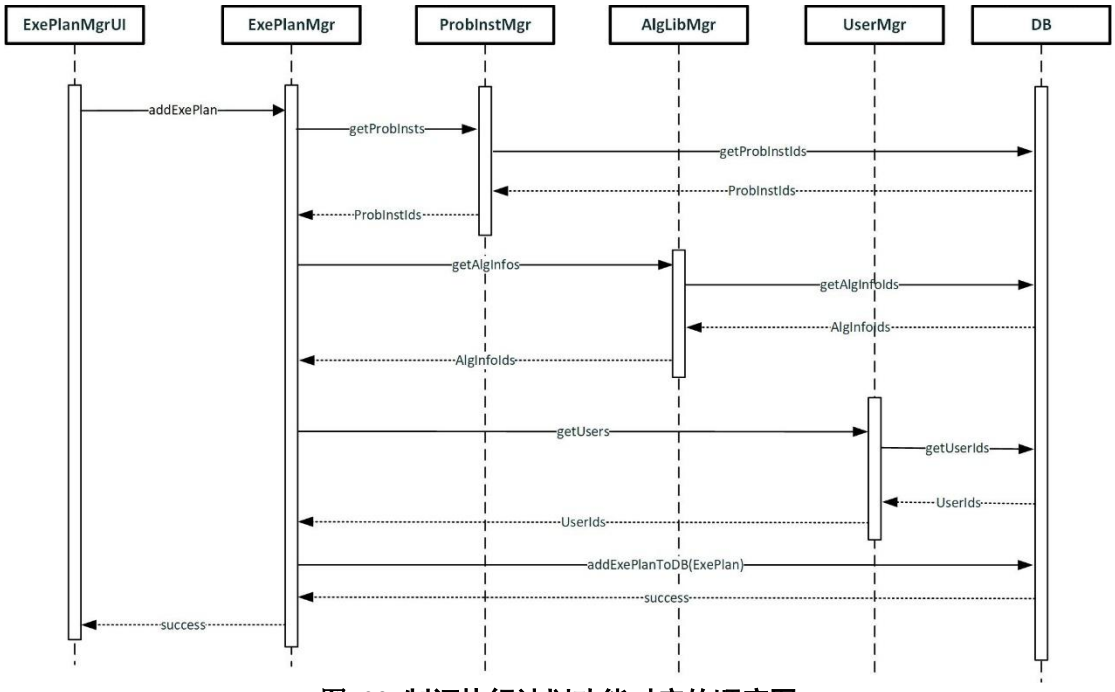

图 23 制订执行计划功能对应的顺序图

（3）api 接口层

Api 接口层面向 Web 应用前端调用需求，设计成问题实例管理、算法库管理、执行计划管理与平台管理四个构件，如图 24 所示。而图 25 给出了可入库算法（由算法设计者负责实现）需实现的接口。

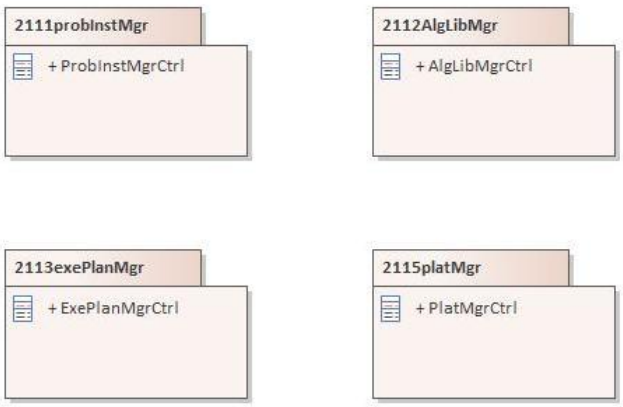

图 24　api 层的构件设计

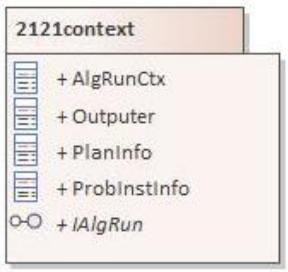

图 25　可入库算法须实现的接口

（4）用户界面层

算法实验平台中问题实例管理、算法库管理、执行计划管理和平台管理的界面设计分别如图 26、图 27、图 28 和图 29 所示。

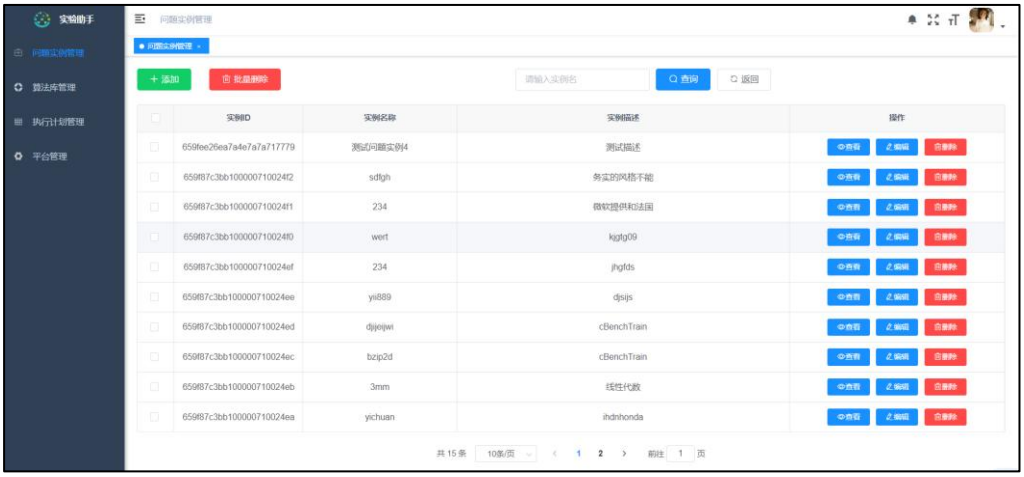

图 26 问题实例管理的界面设计

图 27 算法库管理的界面设计

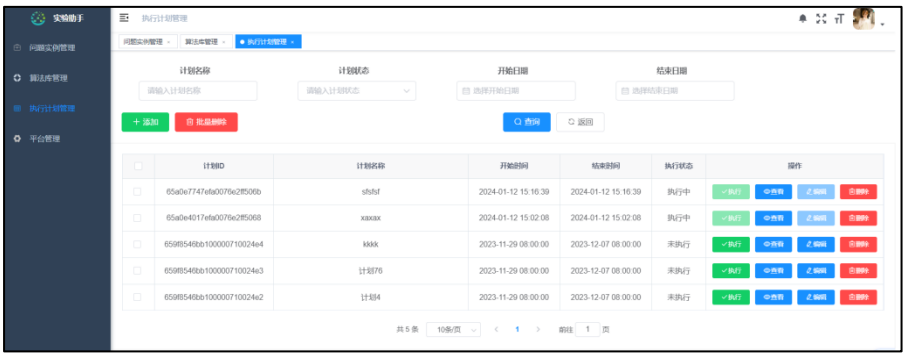

图 28 执行计划管理的界面设计

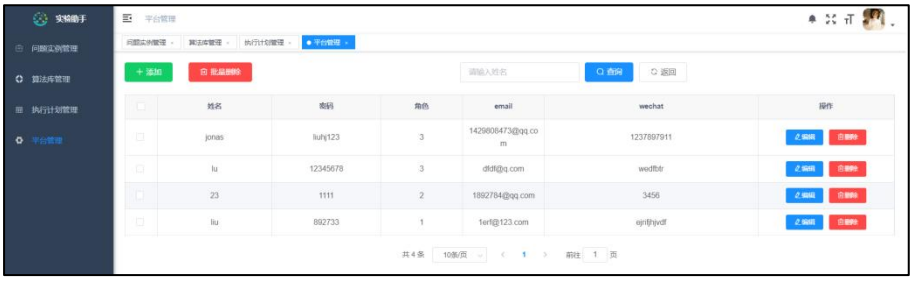

图 29 平台管理的界面设计

4.2.2 开发视图

开发视图将逻辑视图中的设计元素映射为程序单元，并确定程序单元的目录结构。图 30 给出了算法实验平台的程序单元组织结构图。图中 doc 目录用于存放分析模型、设计模型等相关开发文档。

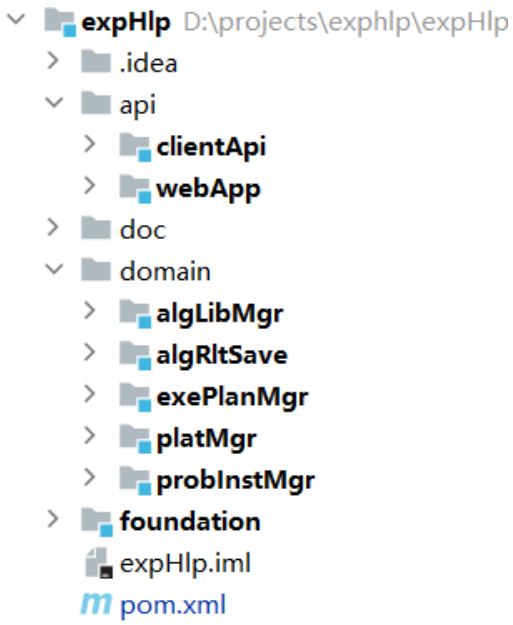

图 30 算法实验平台的程序单元组织结构图

4.2.3 数据视图

软件架构的数据视图主要关注持久数据在宏观层面的存储组织方式。算法实验平台采用 MongoDB 数据库，依据逻辑视图中各子系统中的实体类及其之间关系，设计了 6 个 MongoDB 中的集合。集合元素是键-值对的列表。键的数据类型为字符串，值的类型需要定义。

（1）问题实例管理 probInstMgr 集合

probInstMgr 集合的定义如表 2 所示。

表 2 probInstMgr 集合的定义

键名	值的数据类型	说明
instId	字符串	问题实例 ID
instName	字符串	问题实例名
machineIp	字符串	机器 IP
machineName	字符串	机器名
description	字符串	实例描述
dirName	字符串	实例路径
categoryName	字符串	分类名

（2）算法库管理 algLibMgr 集合

algLibMgr 集合的定义如表 3 和表 4 所示。

表 3 algLibMgr 集合的定义

键名	值的数据类型	说明
algId	字符串	算法 ID
algName	字符串	算法名
algPath	字符串	算法路径
description	字符串	算法的描述
defParas	集合	算法参数,其定义如表 4

表 4 defParas 集合的定义

键名	值的数据类型	说明
paraId	整形	参数 ID
paraName	字符串	参数名
paraType	字符串	参数类型
paraValue	字符串	参数的值
description	字符串	参数的描述

（3）执行计划管理 exePlanMgr 集合

exePlanMgr 集合的定义如表 5、表 6 和表 7 所示。

表 5 exePlanMgr 集合的定义

键名	值的数据类型	说明
planId	字符串	计划 ID
planName	字符串	计划名
probInstIds	数组	问题实例 ID 的列表
algRunInfos	集合	算法运行信息，其定义如表 6 所示
userIds	数组	用户 ID
exeStartTime	长整形	执行开始时间
exeEndTime	长整形	执行结束时间
exeState	整形	计划执行状态
description	字符串	执行计划描述

表 6 algRunInfos 集合的定义

键名	值的数据类型	说明
algRunInfoId	字符串	运行信息 ID
algId	字符串	引用的算法的 Id
algName	字符串	算法名
runNum	整形	运行次数
runParas	集合	运行参数，其定义如表 7

表 7 runParas 集合的定义

键名	值的数据类型	说明
paraId	字符串	运行参数 ID
paraName	字符串	运行参数名
paraType	字符串	运行参数类型
paraValue	字符串	运行参数的值
description	字符串	运行参数的描述

（4）算法结果保存 algRltSave 集合

algRltSave 集合的定义如表 8、表 9 和表 10 所示。

表 8 algRltSave 集合的定义

键名	值的数据类型	说明
planExeResultId	字符串	执行结果 ID
planId	字符串	计划 ID
algId	字符串	算法 ID
runNum	整形	算法运行次序
startTime	长整形	算法执行开始时间
outputTime	长整形	输出时间
genResults	集合	迭代结果，其定义如表 9 所示

表 9 genResults 集合的定义

键名	值的数据类型	说明
algRunResultId	字符串	迭代结果 ID
outTime	长整形	迭代结束时间
probInstId	字符串	问题实例 ID
eachResults	集合	运行结果集

表 10 eachResults 集合的定义

键名	值的数据类型	说明
key	字符串	单次运行结果的键
value	字符串	单次运行结果的值
dataType	字符串	数据类型

（5）用户管理 userMgr 集合

userMgr 集合的定义如表 11 所示。

表 11 userMgr 集合的定义

键名	值的数据类型	说明
userId	字符串	用户 ID
userName	字符串	用户名
role	字符串	用户所属角色
password	字符串	密码
email	字符串	邮箱
wechat	字符串	微信号

4.2.4 物理视图

软件架构的物理视图负责展示软件中各子系统、构件部署到哪些计算结点上运行，以及这些结点之间的网络连接方式。考虑到算法实验平台的可扩展性、性能和可靠性等非功能性需求，采用开源框架 Docker 实现容器（轻量级虚拟机）部署，如图 31 所示。

因容器与容器、容器与客户机之间的通信均采用标准的安全通信协议 Https，为了简洁起，图中略去连接方式。其中：MongoDB、RabbitMQ 和 nacos 为图 16 中工作在基础层的数据库、消息队列和 Web 服务管理构件；WebApp 则对应于界面调用请求的构件，而 Alg1 至 Algn 是由算法设计者实现并由平台部署至容器中的构件。

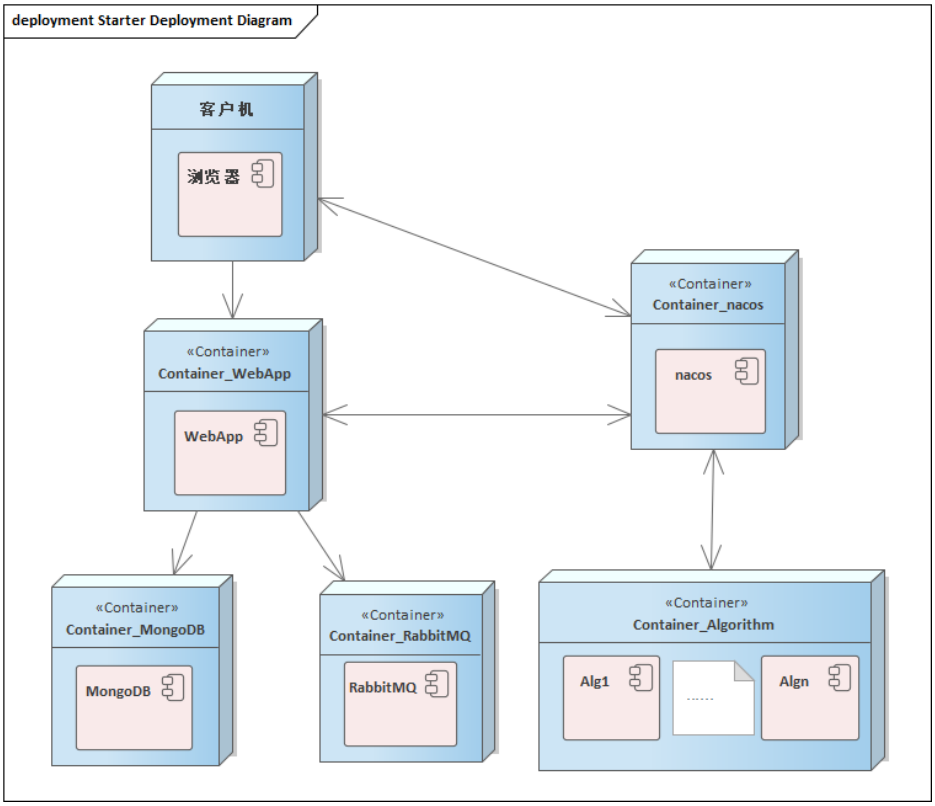

图 31 算法实验平台的物理视图

5 总结

在算法实验平台的用例模型和分析类模型等需求模型基础上，给出平台的软件架构的概念设计和精化设计过程。架构概念设计给出了关键需求辨识和软件架构初创的方法和结果；而架构精化设计则从逻辑视图、开发视图、数据视图和物理视图阐述运用设计知识精化软件架构的过程和结果。

6 英文说明

Software Architecture Design for Algorithm Experiment Platform

Youcong Ni[1], Xin Du[1], Ruliang Xiao[1], Gaolin Chen
[1] (College of Computer and Cyberspace Security, Fujian Normal University, Fuzhou, 350117,China)

Abstract: Building upon the requirement models such as the use case model and analysis class model in the algorithm experiment platform, this document outlines the conceptual design and detailed design process of the platform's software architecture. The conceptual design of the architecture provides methods and outcomes for identifying key requirements and initiating the software architecture. On the other hand, the detailed design of the architecture elucidates the process and outcomes of refining the software architecture using design knowledge, focusing on logical views, development views, data views, and physical views.

Key words: Algorithm Experiment Platform; Software Architecture; Software Architecture Design; Software Architecture Design Process

附参考文献：

[1] C. A. T. Romero, J. H. Ortiz, O. I. Khalaf, W. M. Ortega. Software architecture for planning educational scenarios by applying an agile methodology[J], Int. J. Emerg. Technol. Learn. 2021, 16(8): 132-138.

[2] A. F. Ibrahim, A. S. Attia, B. Asma'M, et al. Evaluation of the online teaching of architectural design and basic design courses case study: College of Architecture at JUST, Jordan[J], Ain Shams Eng. J., 2021,12(2):2345–2353.

[2] 杨楠,李童.案例教学视域下软件工程"一体两翼"教学模式的构建与实践[J], 高等工程教育研究,2020,37(1):177–181.

[3] 陈广智,郑小东,王军. 基于 GPA 变动分析案例的软件工程教学实践[J],计算机教育, 2020, 9(1): 118-122.

[4] 赵明,廖志芳,杨柳等. 软件工程核心课程教学案例库建设以及案例教学实践[J], 计算机教育, 2019, 8(1): 32-35.